```
void loop() {
  digitalWrite(ledPin, HIGH);
  delay(time);
  digitalWrite(ledPin, LOW);
  delay(time);
}

int freeRAM() {
  extern int __heap_start, *__brkval;
  int v;
  return (int) &v - (__brkval == 0 ? (int) &__heap_start : (int) __brkval);
}
```

In this sample code, we tossed in the `freeRAM()` function at the bottom of our sketch and in our `setup()` function we set up serial communications and sent the text string `[memCheck]` followed by the value of free RAM as returned by the `freeRAM()` function. After uploading the source code, open up the Serial Monitor to see the unused amount of RAM. This could be a fairly useful function to have in the toolbox when using large arrays or text strings or just trying to track down some unusual problems with our code.

Armed with a basic understanding of the memory types and structure on the Arduino microcontroller and how we might check the free RAM space in our code, let's look at a couple of the libraries used to access some of the other types of memory space. We might even consider a method for rewriting our project code to save a little memory.

Using Program Memory

Instead of using a character array like we did in Project 8 that gets loaded into RAM memory, we could use the `pgmspace` library that is a part of the standard avr-libc library to keep read-only data stored only in program memory. The benefit to keeping large variables, arrays, or text strings in program memory is that when the program starts, this read-only data is not loaded into the RAM space.

The challenge with this library is that it has not been given the full Arduino treatment, so it's only really worth the hassle of dealing with the more difficult code when we absolutely need to store large chunks of data. It is also a two-step process that requires some extra functions to properly store the data to begin with and then again to read the data in program memory so that we can actually use it. Let's focus this all-too-brief discussion on how we could use program memory to rewrite our project code and save a little memory in the process.

To begin with, we need to let the Arduino compiler know that we want to use the program memory functions that are not a part of the standard Arduino library by specifying the following line at the beginning of our code:

```
#include <avr/pgmspace.h>
```

The easiest way to keep a string in program memory and not in RAM is to use the `PROGMEM` variable qualifier, like in the following example:

```
const char message[] PROGMEM =
"This message is held entirely in flash memory.";
```

This wouldn't work for our project code because we want to create an array of strings, which is not entirely supported by the pgmspace library. To get around this, we could make multiple character arrays that are stored in program memory, as follows: